

개인 기여 및 협업 개선 회고

과목: 문제해결프로그래밍 | 담당교수: 박경신 | 프로젝트: Time Table | 이름: 이근산 | 학번: 32183022

1. 본인이 이번 Group Project에서 담당한 역할과 수행한 기여

이번 프로젝트에서 저는 Time Table의 문제 정의, 서비스 기획, 화면 흐름 설계, 개발 환경 구성, 배포 및 인프라 방향 정리, 테스트 자동화와 검증 흐름 구축을 중심으로 담당하였다. 임베디드 개발 위주로 공부하고 다루어 왔기 때문에 웹서비스 개발은 상대적으로 익숙하지 않았다. 그래서 단순히 코드를 작성하는 것보다 먼저 어떤 문제를 해결할 것인지, 어떤 화면 흐름이 필요한지, 어떤 구조로 구현해야 유지보수할 수 있는지를 정리하는 데 많은 시간을 사용하였다.

특히 프로젝트의 핵심 문제를 “일정을 기록하는 것”이 아니라 “일정, 할 일, 목표, 집중 상태가 흩어져 실행으로 이어지지 않는 것”으로 정의하였다. 개인적으로 일정 관리와 작업 시간을 확보하는 것이 가장 어려웠고, 이 문제를 해결하기 위해 Time Table이 오늘 브리핑, 주간 일정, 목표, 집중 모드를 하나의 작업공간으로 연결하도록 기획하였다.

구현 과정에서는 AI를 적극적으로 활용하였다. 웹 프론트엔드, 백엔드 API, 동기화 구조, 테스트 코드 작성 등 웹서비스 개발의 많은 부분에서 AI를 pair programmer처럼 활용했고, 저는 요구사항 정리, 구현 방향 결정, 결과 검토, 통합, 테스트, 데모 가능 상태 확인에 집중하였다. 이 과정에서 Next.js 기반 프론트엔드, Spring Boot 기반 백엔드, H2/PostgreSQL을 고려한 데이터 구조, mock login과 mock Google sync를 활용한 로컬 데모 환경을 정리하였다.

또한 서비스 내부에 포함되는 AI 기능과 AI harness engineering에도 신경을 많이 썼다. AI가 사용자의 일정을 마음대로 변경하는 것이 아니라, 조정안을 만들고 사용자가 검토할 수 있도록 하는 trust boundary를 설계하려고 했다. AI 제안이 실제로 사용자에게 도움이 되는지 확인하기 위해 테스트 자동화, visual QA, mock data 기반 캡처, 데모 흐름 검증도 함께 진행하였다. 아직 완성도가 충분하다고 보기는 어렵지만, 이번 프로젝트는 AI를 단순 코드 생성 도구로 쓰는 것을 넘어서 AI 기능 자체를 어떻게 검증하고 운영해야 하는지 고민하는 계기가 되었다.

2. 프로젝트 진행 과정에서 개선이 필요하다고 생각한 협업 프로세스

가장 개선이 필요하다고 느낀 부분은 일정 관리와 작업 시간 확보였다. 프로젝트를 진행하면서 해야 할 일은 계속 늘어났지만, 어떤 작업을 언제까지 끝내야 하는지, 어떤 기능을 최종 발표 범위에 포함할지, 어떤 부분은 과감히 줄일지에 대한 기준이 더 명확했으면 좋았다고 생각한다. 특히 웹서비스 프로젝트는 기획, 디자인, 프론트엔드, 백엔드, 데이터베이스, 배포, 테스트가 연결되어 있어 한 부분이 늦어지면 전체 일정이 쉽게 밀렸다.

두 번째로 역할 분담과 커뮤니케이션 방식도 더 체계화될 필요가 있었다. 실제 작업에서는 제가 기획, 설계, 인프라, 테스트 자동화, AI 활용 개발을 많이 맡게 되었고, 그만큼 전체 구조를 제가 계속 파악해야 했다. 하지만 Group Project라면 기능 단위 또는 산출물 단위로 담당자를 더 명확히 정하고, 진행 상황과 막힌 부분을 짧게라도 주기적으로 공유하는 방식이 필요하다.

세 번째로 버전 관리와 AI 활용 결과물 검토 프로세스가 더 필요하다고 느꼈다. AI를 활용하면 짧은 시간에 많은 코드를 만들 수 있지만, 그만큼 변경 범위가 커지고 검토해야 할 내용도 늘어난다. AI가 만든 코드가 실제 요구사항에 맞는지, 기존 구조를 깨뜨리지 않는지, 테스트가 충분한지 확인하는 절차가 없으면 프로젝트 후반에 품질 관리가 어려워진다.

마지막으로 테스트와 데모 준비를 더 일찍 시작했어야 한다. 발표 직전에 화면 캡처, mock data, 배포 링크, 데모 시나리오를 정리하면 예상하지 못한 오류가 생겼을 때 대응 시간이 부족하다. 이번

프로젝트에서도 발표 자료와 보고서를 정리하면서 데모 가능 범위와 남은 작업을 다시 구분해야 했고, 이 부분은 다음 프로젝트에서 더 일찍 관리해야 할 부분이라고 생각한다.

3. 개선점을 보완하기 위한 구체적인 방안

첫째, 프로젝트 초기에 milestone을 더 작게 나누고, 각 milestone마다 산출물을 명확히 정해야 한다. 예를 들어 1주차에는 문제 정의와 화면 흐름, 2주차에는 핵심 API와 mock data, 3주차에는 대시보드와 일정 화면, 4주차에는 집중 모드와 테스트 자동화, 마지막 주에는 발표 자료와 데모 안정화처럼 단계별 완료 기준을 정한다. 이렇게 하면 막연히 "기능 개발"을 하는 것이 아니라 발표 가능한 상태를 기준으로 진행할 수 있다.

둘째, 역할 분담은 사람 단위가 아니라 책임 영역 단위로 나누는 것이 좋다. 기획/문서, 프론트엔드 화면, 백엔드 API, 데이터/동기화, 테스트/배포, 발표 자료처럼 영역을 나누고 각 영역의 owner를 정한다. owner는 모든 코드를 혼자 작성한다는 의미가 아니라, 해당 영역의 요구사항, 진행 상황, 남은 문제를 책임지고 정리하는 역할이다.

셋째, 커뮤니케이션은 짧고 자주 하는 방식이 필요하다. 긴 회의보다 GitHub Issues, Projects board, 짧은 daily log를 활용해 "어제 한 일, 오늘 할 일, 막힌 점"을 남기면 좋다. 특히 일정 관리가 어려운 경우에는 개인이 머릿속으로만 계획을 갖고 있기보다, 작은 task 단위로 쪼개서 board에 올리고 마감일을 표시하는 방식이 도움이 된다.

넷째, AI 활용 프로세스를 명시해야 한다. AI를 사용해 코드를 작성할 때는 단순히 결과물을 붙여 넣는 것이 아니라, 어떤 요구사항을 넣었는지, 어떤 파일이 바뀌었는지, 어떤 테스트로 확인했는지를 기록해야 한다. 또한 AI가 만든 코드는 최소한 typecheck, build, unit test, e2e 또는 visual QA 중 하나 이상의 검증을 통과한 뒤 main branch에 반영하는 규칙이 필요하다.

다섯째, AI harness engineering을 프로젝트의 별도 품질 관리 항목으로 다루고 싶다. 서비스 안의 AI 기능은 "그럴듯한 답변"만으로는 충분하지 않다. 입력 데이터, 예상 출력, 금지해야 할 행동, 사용자의 승인 여부, 실패했을 때의 fallback을 테스트 케이스로 만들어야 한다. 다음 프로젝트에서는 AI 기능을 개발할 때 prompt, test fixture, evaluation log, regression test를 함께 관리하여 AI 기능의 신뢰도를 더 체계적으로 높이고 싶다.

마지막으로 발표와 제출 준비는 개발 마지막 단계가 아니라 프로젝트 초반부터 병행해야 한다. 데모 시나리오, screenshot 후보, 제출 체크리스트, 공개 GitHub에서 제외할 secret 목록을 미리 만들어 두면 발표 직전에 급하게 정리하는 부담을 줄일 수 있다. 이번 경험을 통해 일정 관리와 AI 활용이 프로젝트 성공에 큰 영향을 준다는 것을 느꼈고, 앞으로는 AI를 적극적으로 활용하되 그 결과를 검증하는 harness와 협업 프로세스를 함께 강화해야겠다고 생각했다.